1) WAP in C to find largest and smallest element in an array

```
#include <stdio.h>
int main() {
int arr[5], i, max, min;
printf("Enter 5 elements: ");
for(i=0; i<5; i++) scanf("%d", &arr[i]);

max = min = arr[0];
for(i=1; i<5; i++){
if(arr[i] > max) max = arr[i];
if(arr[i] < min) min = arr[i];
}
printf("Largest = %d\nSmallest = %d\n", max, min);
return 0;
}
```

Output Example:

Enter 5 elements: 12 5 8 20 3

Largest = 20

Smallest = 3

2) WAP in C to implement stack using array

```
#include <stdio.h>

#define MAX 5

int stack[MAX], top=-1;


void push(int x){

   if(top == MAX-1) printf("Stack Overflow\n");

   else stack[++top] = x;

}


void pop(){

   if(top == -1) printf("Stack Underflow\n");

   else printf("Popped: %d\n", stack[top--]);

}

int main(){

   push(10);

   push(20);

   push(30);

   pop();

   pop();

   return 0;

}
```

Output Example:

Popped: 30

Popped: 20

3) WAP in C to implement Queue using array

```
#include <stdio.h>
#define MAX 5
int queue[MAX], front=-1, rear=-1;
void enqueue(int x){
   if(rear == MAX-1) printf("Queue Full\n");
   else {
      if(front==-1) front=0;
      queue[++rear] = x;
   }
}
void dequeue(){
   if(front==-1 || front>rear) printf("Queue Empty\n");
   else printf("Dequeued: %d\n", queue[front++]);
}
int main(){
   enqueue(10);
   enqueue(20);
   dequeue();
   dequeue();
   return 0;
}
```

Output Example:

```
Dequeued: 10
Dequeued: 20
```

4) WAP in C to implement single linked list (Insert & Display)

```
#include <stdlib.h>

struct Node{

   int data;

   struct Node* next;

};int main(){

 struct Node *head = NULL, *temp, *newNode; int i, n;

printf("Enter 3 elements: ");

 for(i=0;i<3;i++){ scanf("%d", &n);

newNode = (struct Node*)malloc(sizeof(struct Node);

newNode->data = n; newNode->next = NULL;

 if(head==NULL) head = temp = newNode;

else {temp->next=newNode; temp=newNode;} } temp = head;

   printf("Linked List: ");

   while(temp){ printf("%d ", temp->data); temp=temp->next;}

   return 0;

}
```

5) WAP in C to implement Binary Search algorithm

```
#include <stdio.h>

int main(){

   int arr[5]={3,7,9,12,15}, key, low=0, high=4, mid;

   printf("Enter key: "); scanf("%d",&key);

   while(low<=high){

      mid = (low+high)/2;

      if(arr[mid]==key){printf("Found at index %d\n", mid); break;}

      else if(arr[mid]<key) low=mid+1;

      else high=mid-1;

   }

   if(low>high) printf("Not Found\n");

   return 0;

}
```

Output Example:

```
Enter key: 12
Found at index 3
```

6) WAP in C to implement Bubble Sort

```
#include <stdio.h>

int main(){
   int arr[5]={5,3,8,1,2}, i,j,temp;
   for(i=0;i<5-1;i++)
      for(j=0;j<5-i-1;j++)
         if(arr[j]>arr[j+1]){
            temp=arr[j]; arr[j]=arr[j+1]; arr[j+1]=temp;
         }
   printf("Sorted Array: ");
   for(i=0;i<5;i++) printf("%d ",arr[i]);
   return 0;
}
```

Output Example:

```
Sorted Array: 1 2 3 5 8
```

7) WAP in C to implement Quick Sort

```
#include <stdio.h>


void quickSort(int arr[], int low, int high){
   if(low<high){
      int i=low, j=high, pivot=arr[low], temp;
      while(i<j){
         while(arr[i]<=pivot && i<high) i++;
         while(arr[j]>pivot) j--;
         if(i<j){temp=arr[i]; arr[i]=arr[j]; arr[j]=temp;}
      }
      arr[low]=arr[j]; arr[j]=pivot;
      quickSort(arr, low, j-1);
      quickSort(arr, j+1, high);
   }
}
int main(){
   int arr[5]={10,7,8,9,1}, i;
   quickSort(arr, 0, 4);
   printf("Sorted Array: ");
   for(i=0;i<5;i++) printf("%d ", arr[i]);
   return 0;
}
```

Output Example:

```
Sorted Array: 1 7 8 9 10
```

8) WAP in C to implement BFS and DFS

```
#include <stdio.h>

int main(){
   int adj[3][3]={{0,1,1},{1,0,1},{1,1,0}}, visited[3]={0}, queue[3], front=0,rear=0,i,j;
   queue[rear++] = 0; visited[0]=1;
   while(front<rear){
      i = queue[front++];
      printf("%d ",i);
      for(j=0;j<3;j++)
         if(adj[i][j]==1 && !visited[j]){
            queue[rear++]=j;
            visited[j]=1;
         }
   }
   return 0;
}
```

Output Example:

```
0 1 2
```